

Proyecto final entrega N°1

Luis Miguel Garcia Villanueva

Código: 1092459065

Juan David Arias Sanchez

Código:1092853032



**UNIVERSIDAD
DEL QUINDÍO**

Profesora:

Carolina Londoño Idarraga

Universidad del Quindío

Facultad de Ingeniería

Programa de Ingeniería de Sistemas y Computación

Teorema de diseño organizacional

Grupo 02D

Armenia, Quindío, 2026

1. PENSAMIENTO COMPUTACIONAL (RF-043)

Abstracción:

- ¿Qué se solicita finalmente?

Desarrollar una programa de gestión de eventos utilizando Java, JavaFX y a su vez patrones de diseño que permita a Usuarios explorar eventos, seleccionar asientos por zona, comprar entradas, agregar servicios adicionales (VIP, seguro, merchandising, parqueadero), pagar y recibir comprobantes. A administradores: gestionar el catálogo completo (eventos, recintos, zonas, asientos), controlar disponibilidad y registrar incidencias, por último el programa debe generar reportes exportables en formato PDF o CSV.

- ¿Qué información es relevante?

La información relevante es:

- Usuario: idUsuario, nombre, correo, teléfono, métodos de pago.
- Evento: idEvento, nombre, categoría, ciudad, fecha, estado (Pausado/Cancelado/Finalizado), políticas.
- Recinto: idRecinto, nombre, dirección, zonas.
- Zona: idZona, nombre, capacidad, precio base.
- Asiento: idAsiento, fila, número, estado (Vendido/Bloqueado).
- Compra: idCompra, usuario, evento, total, estado (Confirmada/Cancelada/Reembolsada/Incidencia).
- Entrada: idEntrada, zona, asiento, precio, estado (Usada/Anulada).
- Pago: idPago, monto, método, fecha.
- Incidencia: idIncidencia, tipo, descripción, entidad afectada.

Nota: Un asiento solo puede pertenecer a una Zona; una Zona solo a un Recinto. Una Compra puede tener N Entradas y M Servicios Adicionales. El precio final de una Entrada puede incluir decoradores (carga VIP, carga preferencial, carga por servicio). No se puede comprar un asiento Vendido o Bloqueado. Cancelaciones y reembolsos siguen políticas definidas por evento.

- ¿Cómo se agrupa la información relevante? (clases y estructura)

La información relevante se clasifica en:

Entidades con atributos estados y relaciones: Usuario, Evento, Recinto, Zona, Asiento, Compra, Entrada, Pago, Tarifa, Incidencia, ServicioAdicional.

Lógica de negocio y validaciones: EventoService, CompraService, AsientoService, PagoService, UsuarioService, ReporteService.

Repositorios (guardar y buscar data): EventoRepository, CompraRepository, AsientoRepository, UsuarioRepository.

Patrones creacionales: EventoFactory, CompraBuilder, DatabaseManage (Singleton).

Patrones estructurales: EntradaDecorator, VIPDecorator, SeguroDecorator, MerchandisingDecorator, PagoAdapter, ProxyServicioExterno.

Patrones comportamentales: PagoStrategy, EfectivoStrategy, TarjetaStrategy, PSEStrategy, ObservadorEvento, NotificadorUsuario, CompraStateManager.

Interfaz gráfica: LoginController, EventoListController, CompraController, AdminDashboardController, MetricasController.

Reporte: ReporteCSV, ReportePDF, GeneradorReporte.

- ¿Qué funcionalidades se solicitan?

Para el usuario: registro, búsqueda filtrada de eventos, selección visual de asientos, personalización de compra (servicios extra), pago simulado e historial. Para el administrador: gestión completa (CRUD) del catálogo, administración de disponibilidad, manejo de cancelaciones/reembolsos e incidencias, y un panel analítico gráfico.

Descomposición:

- ¿Cómo se distribuyen las funcionalidades?

Las responsabilidades se dividen bajo el Principio de Responsabilidad Única (SRP). La capa UI (Controladores) solo captura eventos de la pantalla; delega a la capa de Servicios, que contiene toda la lógica de negocio y transiciones de estado; la cual a su vez consulta a la capa de Repositorios, responsable del acceso a las colecciones de datos en memoria.

- ¿Qué debo hacer para probar las funcionalidades?

Puedo clasificar las pruebas de la siguiente manera:

- Pruebas de Repositorio (Datos iniciales): Verificar que al iniciar la aplicación (Data Initializer) se carguen en memoria los 3 usuarios, 5 eventos, y más de 50 asientos requeridos.
- Pruebas de Servicio (Lógica y estados): Comprobar transiciones puras. Por ejemplo, si el administrador cambia un evento de "Publicado" a "Cancelado", verificar que el patrón Observer dispare la cancelación de todas las compras asociadas. Probar el Decorator creando una compra base y añadiéndole VIP (\$50k) y seguro (\$15k) para asegurar que el cálculo total en la capa de servicio sea exacto.
- Pruebas de Interfaz y Concurrencia: Validar que los botones en JavaFX llamen correctamente a los controladores. Simular que dos usuarios intentan comprar el mismo asiento al mismo tiempo para comprobar que el CompraService rechaza la segunda petición, manteniendo la consistencia.
- Pruebas de Salida (Reportes): Ejecutar descargas desde el perfil de usuario y administrador con diferentes rangos de fechas, verificando que los archivos CSV y PDF se creen correctamente en el disco usando las estrategias de pago y reportes correspondientes.

Reconocimiento de patrones:

- ¿Qué puedo reutilizar de la solución de otros problemas?

Decorator genérico: una interfaz IEntradaDecorable y una clase base AbstractEntradaDecorator permiten añadir cualquier cargo adicional (VIP, seguro, merchandising, parqueadero) sin duplicar código.

Strategy de pago: la interfaz PagoStrategy es reutilizable para cualquier método de pago actual o futuro (criptomonedas, puntos, etc.).

Observer genérico: el mecanismo de suscripción/notificación aplica tanto para cambios de estado de Evento como de Compra.

Factory genérica: EventoFactory y CompraFactory pueden extenderse para nuevos tipos de eventos o compras sin modificar el código cliente.

DatabaseManager Singleton: punto único de acceso a los datos en memoria, reutilizado por todos los repositorios.

ReporteStrategy: el mecanismo de generación de reportes puede intercambiar entre CSV y PDF sin modificar el código cliente.

2. JUSTIFICACIÓN DE LOS PATRONES UTILIZADOS (RF-049, RF-050, RF-051)

Patrones creacionales:

- Singleton

Requisitos que resuelve:

RF-045 (datos de prueba inicializados), RF-015 (asientos y disponibilidad), RF-016 (gestión de compras)

Problema que resuelve:

La aplicación necesita un único punto de acceso a los repositorios de datos en memoria (usuarios, eventos, compras, asientos). Si múltiples instancias del repositorio coexisten, los datos quedarían inconsistentes (un asiento podría aparecer disponible en un repositorio y vendido en otro, generando venta doble).

Por qué este patrón y no otro:

Singleton garantiza exactamente una instancia de DatabaseManager compartida por todos los servicios y así centralizar la persistencia de los datos en memoria.

- Factory Method

Requisitos que resuelve:

RF-013 (crear eventos por categoría: concierto, teatro, conferencia), RF-034 (crear compras nuevas), RF-038 (generar entradas).

Problema en el contexto:

Crear un Evento requiere lógica diferente según su categoría: un concierto tiene zonas VIP/General, un teatro tiene butacas numeradas, una conferencia puede ser de aforo libre. Si esta lógica vive en el cliente, cada pantalla de administración debe conocer todos los tipos, violando OCP.

Por qué este patrón y no otro:

Factory Method permite que EventoFactory decida qué subclase o configuración crear según la categoría, sin exponer la lógica al cliente.

- Builder

Requisitos que resuelve:

RF-006 (crear/modificar compra), RF-009 (agregar servicios adicionales), RF-035 (modificar compra antes de pagar).

Problema en el contexto:

Construir una Compra es un proceso en pasos: primero se seleccionan entradas, luego se agregan servicios adicionales opcionales (VIP, seguro, merchandising, parqueadero), luego se confirma el pago. El objeto Compra tiene muchos atributos opcionales por ende se elige builder para evitar un constructor inmenso.

Por qué este patrón y no otro:

Builder permite construir la Compra paso a paso, añadiendo ítems y servicios de forma gradual.

Patrones Estructurales:

- Decorator

Requisitos que resuelve:

RF-009 (servicios adicionales: VIP, seguro, merchandising, parqueadero), RF-007 (pago con precio calculado), RF-029 (precio base por zona).

Problema en el contexto:

El precio de una entrada puede variar según los servicios adicionales que el usuario elija. Con herencia, necesitaríamos una subclase por cada combinación posible (EntradaVIP, EntradaConSeguro, EntradaVIPConSeguroConMerchandising, etc).

Por qué este patrón y no otro:

Decorator permite añadir responsabilidades (cargos adicionales) a un objeto dinámicamente, en tiempo de ejecución, apilando decoradores.

- Proxy

Requisitos que resuelve:

RF-005 (selección de asientos según disponibilidad), RF-015 (habilitar/bloquear/liberar asientos), RF-017 (registrar incidencias automáticamente)

Problema en el contexto:

El acceso al servicio de gestión de asientos debe ser controlado: solo el administrador puede bloquear asientos, y cualquier operación sobre asientos debe quedar registrada (auditoría) para detectar incidencias como las reservas dobles. Si el control de acceso y el logging se mezclan en el Service principal, violaría SRP.

Por qué este patrón y no otro:

El proxy y el logging interceptan las llamadas a AsientoService, verifican el rol del usuario antes de ejecutar operaciones y registra automáticamente cada cambio de estado.

- Adapter

Requisitos que resuelve:

RF-007 (pagar la compra), RF-046 (generador de reportes CSV/PDF), RF-021 (métodos de pago simulados)

Problema en el contexto:

La generación de reportes usa dos bibliotecas con APIs completamente distintas: CSV y PDF. El código cliente no debería depender de las APIs específicas de cada biblioteca; si se cambia la biblioteca, el cliente no debería modificarse.

Por qué este patrón y no otro:

Adapter convierte la interfaz de una biblioteca (incompatible) en la interfaz que el cliente espera.

Patrones comportamentales:

- Strategy

Requisitos que resuelve:

RF-007 (pagar la compra), RF-021 (métodos de pago simulados: efectivo, tarjeta de crédito, PSE)

Problema en el contexto:

El proceso de pago varía completamente según el método elegido: efectivo solo registra el monto, tarjeta de crédito válida número y CVV, PSE genera una referencia de transacción. Si se usan (if/else o switch en CompraService, agregar un nuevo método de pago obliga a modificar la clase, violando OCP.

Por qué este patrón y no otro:

Strategy encapsula cada algoritmo de pago en su propia clase, intercambiable en tiempo de ejecución. El cliente (CompraService) trabaja solo con la interfaz IPagoStrategy.

- Observer

Requisitos que resuelve:

RF-008 (visualizar estado de la compra), RF-017 (registrar incidencias y cambios de estado), RF-024 (publicar/pausar/cancelar eventos), RF-013 (cambios de estado de evento notifican a compradores).

Problema en el contexto:

Cuando un administrador cancela un evento, todos los usuarios con compras activas deben ser notificados y sus compras deben transicionar a estado 'Cancelada'. Si EventoService llama directamente a CompraService y NotificacionService, crea acoplamiento entre capas y viola el principio de dependencia.

Por qué este patrón y no otro:

Observer desacopla el publicador (Evento) de los suscriptores (CompraService, NotificacionService, IncidenciaService). Cada suscriptor reacciona de forma independiente al cambio.

- State

Requisitos que resuelve:

RF-008 (estados de compra: Creada, Pagada, Confirmada, Cancelada, Reembolsada, Incidencia), RF-036 (cancelar según reglas/políticas), RF-032 (estados de asiento)

Problema en el contexto:

Una Compra tiene 6 estados posibles y las operaciones válidas cambian según el estado: solo se puede pagar una compra en estado Creada, solo se puede reembolsar una compra en estado Pagada o Confirmada (según las políticas), etc. Con condiciones en CompraService, la clase crece indefinidamente con cada nuevo estado o transición.

Por qué este patrón y no otro:

State encapsula el comportamiento de cada estado en su propia clase, haciendo que la Compra delegue las operaciones al estado actual. Cada estado sólo permite las transiciones válidas.